University of Toronto Scarborough
CSCC24 2023 Summer Midterm Test
Duration - 1 hour 30 minutes
Aids allowed: 1 aid sheet (letter size or A4, double-sided, don't hand in).

1:	/12
2:	/12
3:	/14
4:	/12
5:	/ 6
6:	/ 8
Total:	/64

There are blank pages for scratch work at the end.

1. Miscellaneous short questions.

   (a) [3 marks] Here is a function, its type, and a test case I tried:

```
scanr :: (a -> b -> b) -> b -> [a] -> [b]
-- My test case:
--   scanr (\x xs -> x:xs) [100] [1,2]
-- = [[1,2,100],[2,100],[100]]
```

   Predict the answer of `scanr (+) 10 [1,2]`.

   (b) [3 marks] The library function `lookup` performs linear search on a key-value list, as shown below. Express it in terms of `foldr` (reminder in the Appendix), i.e., define `z` and `op` in the following.

```
lookup _ [] = Nothing
lookup x ((k,v) : kvs) | x == k = Just v
                       | otherwise = lookup x kvs

lookup x = foldr op z  where
```

   (c) [3 marks] Rewrite the parser `liftA2 (+) natural natural` using `(>>=)` and `pure` to replace `liftA2`. (Reminders in the Appendix.)

   (d) [3 marks] Rewrite the parser `natural >>= \n -> pure (n + 1)` using `fmap` to replace `(>>=)` and `pure`. (Reminders in the Appendix.)

2. The Unix PATH environment variable stores a lot of directory names, separated by colons, e.g.,

   ```
   /bin:/usr/bin:/usr/games:/home/myself/bin:/usr/local/bin:/usr/local/games
   ```

   Trying to delete anything from it is super annoying. Let's write some Haskell to help!

   We assume no two consecutive colons. We assume no leading or trailing colons in the original input (but it can still happen during recursion).

   (a) [6 marks] `splitOnce` should split a string at the first colon, e.g.,

   ```
   splitOnce "foo:bar:abc" = ("foo", "bar:abc")
   splitOnce ":bar:abc" = ("", "bar:abc") -- hint hint.
   splitOnce "foo" = ("foo", "")
   splitOnce "" = ("", "")
   ```

   (b) [3 marks] `split` should split a string by colons (use `splitOnce` to help), e.g.,

   ```
   split "foo:bar:abc" = ["foo", "bar", "abc"]
   split "foo" = ["foo"]
   split "" = []
   ```

(c) [1 mark] `delete` should delete a string from a list, e.g.,

```
delete "bar" ["foo", "bar", "abc", "bar"] = ["foo", "abc"]
delete "dne" ["foo", "bar", "abc", "bar"] = ["foo", "bar", "abc", "bar"]
```

Implement it using the library function `filter`. (The Appendix has a reminder.)

```
delete s = filter
```

(d) [2 marks] `glue` should be a kind of inverse of `split`. (The `++` operator concatenations two lists/strings.) Examples:

```
glue ["foo", "bar", "abc"] = "foo:bar:abc"
glue ["foo"] = "foo"
glue [] = ""
```

3. Lazy evaluation. The following definitions are given:

```
[]     ++ ys = ys
(x:xs) ++ ys = x : (xs ++ ys)

head (x:xs) = x
```

(a) [5 marks] Show the lazy evaluation steps of the following until you get the answer (a number):

```
   head ( ( (1:[]) ++ (2:[]) ) ++ (3:[]) )
->
```

(b) [2 marks] In general, how much time does the following take? You can give a big-Θ answer.

```
head (...(([1] ++ [2]) ++ [3]) ++ ...) ++ [n]
```

i.e., using `(++)` to combine `[1]` to `[n]`, but with left association.

(c) [5 marks] The following definitions are given:

```
f i = i : f (i + 1)
s3 (x : y : z : _) = [x, y, z]
```

Show lazy evaluation steps of the following until `s3` produces the answer list. It means evaluating `f 0` until the first 3 list nodes emerge. But elements are still unevaluated. Also, show sharing.

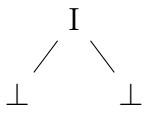
```
   s3 (f 1)
->
```

(d) [2 marks] If `f 0` is evaluated until the first n nodes emerge, without evaluating the number elements, why does it take only $O(n)$ space?

4. Successive approximations and induction. The following definitions are given:

```
data Bit = O | I   -- think 0 and 1, but need uppercase letters
data T = N T Bit T
tL = N tL O tR
tR = N tL I tR
```

(a) [5 marks] Calculate the approximations tL_3 and tR_3. It is probably best to draw pictures instead, e.g., tR_1 looks like:



(b) [7 marks] Given `tL` and `tR` again, and one more definition:

```
tL = N tL O tR
tR = N tL I tR
w [] (N _ a _ ) = [a]
w (O : bt) (N lt a rt) = a : w bt lt
w (I : bt) (N lt a rt) = a : w bt rt
```

Prove: For all finite lists `bs`,
`w bs tL = O : bs` and `w bs tR = I : bs`
by structural induction on `bs`. In the induction step, just work out the `(O : bt)` case, skip the `(I : bt)` case.

5. In statistics, the sums $\sum x_i$, $\sum {x_i}^2$, and the sample size n are important in defining the sample mean and the sample variance. We will set up a Monoid instance so that `foldMap` can compute them. We define:

```
data S = MkS Double Double Int
```

The goal is that with a suitable function `f :: Double -> S`, this will hold:

$$\mathit{foldMap}\ f\ [x_1, ..., x_n] = \mathit{MkS}\ \left(\sum_{i=1}^{n} x_i\right)\ \left(\sum_{i=1}^{n} {x_i}^2\right)\ n$$

The Appendix has reminders for `Semigroup`, `Monoid`, and `foldMap` in `Foldable`.

(a) [3 marks] Make `S` an instance of `Semigroup` and `Monoid` to help fulfill the goal.

```
instance Monoid S where
  -- mempty :: S

  mempty =

instance Semigroup E where
  -- (<>) :: S -> S -> S
```

(b) [3 marks] Implement `f` to complete the goal.

6. Why does C require parentheses around `if` tests? E.g., "`if (x) expr;`". Only one way to find out: Suppose they were not required, then the grammar would look like this:

```
S ::= "if" E E ";"
E ::= "++" E | P
P ::= P "++" | V
V ::= "x" | "y"
```

(a) [6 marks] Show two different parse trees for "`if x ++ y;`".

(b) [2 marks] Why doesn't Haskell require parentheses around `if` tests?

(End of questions.)

Blank page for sketch work.

Blank page for sketch work.

Appendix

```
data Maybe a = Nothing | Just a

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op z [] = z
foldr op z (x : xt) = op x (foldr op z xt)

filter :: (a -> Bool) -> [a] -> [a]
-- Example:
filter odd [3,4,5,6] = [3,5]

class Semigroup a where
  (<>) :: a -> a -> a

class Semigroup a => Monoid a where
  mempty :: a

class Foldable f where
  foldMap :: Monoid m => (a -> m) -> f a -> m
  -- and others

-- Parsing library.
fmap :: (a -> b) -> Parser a -> Parser b
liftA2 :: (a -> b -> c) -> Parser a -> Parser b -> Parser c
pure :: a -> Parser a
(>>=) :: Parser a -> (a -> Parser b) -> Parser b
natural :: Parser Integer
```